Code Search based on Context-aware Code Translation

Weisong Sun
weisongsun@smail.nju.edu.cn
State Key Laboratory for Novel Software Technology
Nanjing University, China

Chunrong Fang*
fangchunrong@nju.edu.cn
State Key Laboratory for Novel Software Technology
Nanjing University, China

Yuchen Chen
yuc.chen@outlook.com
State Key Laboratory for Novel Software Technology
Nanjing University, China

Guanhong Tao
taog@purdue.edu
Purdue University
West Lafayette, Indiana, USA

Tingxu Han
hantingxv@163.com
School of Information Management
Nanjing University, China

Quanjun Zhang
quanjun.zhang@smail.nju.edu.cn
State Key Laboratory for Novel Software Technology
Nanjing University, China

ABSTRACT

Code search is a widely used technique by developers during software development. It provides semantically similar implementations from a large code corpus to developers based on their queries. Existing techniques leverage deep learning models to construct embedding representations for code snippets and queries, respectively. Features such as abstract syntactic trees, control flow graphs, etc., are commonly employed for representing the semantics of code snippets. However, the same structure of these features does not necessarily denote the same semantics of code snippets, and vice versa. In addition, these techniques utilize multiple different word mapping functions that map query words/code tokens to embedding representations. This causes diverged embeddings of the same word/token in queries and code snippets. We propose a novel context-aware code translation technique that translates code snippets into natural language descriptions (called translations). The code translation is conducted on machine instructions, where the context information is collected by simulating the execution of instructions. We further design a shared word mapping function using one single vocabulary for generating embeddings for both translations and queries. We evaluate the effectiveness of our technique, called TranCS, on the CodeSearchNet corpus with 1,000 queries. Experimental results show that TranCS significantly outperforms state-of-the-art techniques by 49.31% to 66.50% in terms of MRR (mean reciprocal rank).

CCS CONCEPTS

• **Software and its engineering → Search-based software engineering**.

KEYWORDS

code search, deep learning, code translation

ACM Reference Format:
Weisong Sun, Chunrong Fang, Yuchen Chen, Guanhong Tao, Tingxu Han, and Quanjun Zhang. 2022. Code Search based on Context-aware Code Translation. In *44th International Conference on Software Engineering (ICSE '22), May 21–29, 2022, Pittsburgh, P A, USA.* ACM, New York, NY, USA, 13 pages. https://doi.org/10.1145/3510003.3510140

1 INTRODUCTION

Software development is usually a repetitive task, where same or similar implementations exist in established projects or online forums. Developers tend to search for those high-quality implementations for reference or reuse, so as to enhance the productivity and quality of their development [4, 5, 16]. Existing studies [5, 56] show that developers often spend 19% of their time on finding reusable code examples during software development. Code search (CS) is an active research field [7, 18, 41, 49, 54, 56, 62–64, 67], which aims at designing advanced techniques to support code retrieval services. Given a query by the developer, CS retrieves code snippets that are related to the query from a large-scale code corpus, such as GitHub [17] and Stack Overflow [26]. Figure 1 shows an example. The query "how to calculate the factorial of a number" in Figure 1(a) is provided by the developer, which is usually a short natural language sentence describing the functionality of the desired code snippet [35]. The method/function [27, 48, 56, 62] in Figure 1(b) is a possible code snippet that satisfies the developer's requirement.

how to calculate the factorial of a number.

(a) A Query

```
1  public long factorial (int number) {
2    long factorial = 0;
3    int i = 0;
4    for (; i <= number; i++) {
5      factorial = factorial * i;
6    }
7    return factorial;
9  }
```

(b) A Code Snippet s_1

Figure 1: An Example of Query and Code Snippet

Existing CS techniques can be categorized into *traditional methods* that use keyword matching between queries and code snippets

*Corresponding author.

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than ACM must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.
ICSE '22, May 21–29, 2022, Pittsburgh, PA, USA
© 2022 Association for Computing Machinery.
ACM ISBN 978-1-4503-9221-1/22/05…$15.00
https://doi.org/10.1145/3510003.3510140

such as information retrieval-based code search [4, 27, 40, 51, 54] and query reformulation-based code search [22, 31, 36, 37, 48, 52], and *deep learning methods* that encode queries and code snippets into embedding representations capturing semantic information. Traditional methods simply treat queries and code snippets as plain texts, and retrieve query-related code snippets by only looking at matched keywords. They fail to capture the semantics of both query sentences and code snippets. Deep learning (DL) methods transform input queries and code snippets into embedding representations. Specifically, for a given query, all the words in the query sentence are first represented as word embeddings and then fed to a DL model to produce a query embedding [18, 56]. For a code snippet, multiple aspects are extracted as features, such as tokens, abstract syntactic trees (ASTs), and control flow graphs (CFGs). These features are transformed into corresponding embeddings and processed by another DL model to produce a code embedding [12, 18, 56, 64, 67]. The code search task is hence to find similar pairs between query embeddings and code embeddings. While *DL methods* surpass *traditional methods* in capturing the semantics of queries and code snippets, their performances are still limited due to the insufficiency of encoding semantics and the embedding discrepancy between queries and code snippets. Existing techniques miss either data dependencies among code statements like MMAN [62] or control dependencies such as DeepCS [18], CARLCS-CNN [56], and TabCS [64]. Furthermore, the embedding representations of code snippets are largely different from those of query sentences written in natural language, causing semantic mismatch during the code search task. For example, MMAN [62] uses different word mapping functions (that map a word or token to an embedding representation) to encode queries, and tokens, ASTs, and CFGs in code snippets. For the widely used word `length` in both queries and code snippets, the embedding representations are different in those word mapping functions, leading to poor code search performance as we will discuss in Section 3 and experimentally show in Section 5.2.2.

We propose a novel context-aware code translation technique that translates code snippets into natural language descriptions (called translations). Such a translation can bridge the representation discrepancy between code snippets (in programming languages) and queries (in natural language). Specifically, we utilize a standard program compiler and a disassembler to generate the instruction sequence of a code snippet. However, the context information such as local variables, data dependency, etc., are missed from the instruction sequence. We hence simulates the execution of instructions to collect those desired contexts. A set of pre-defined translation rules are then used to translate the instruction sequence and contexts into translations. Such a code translation is context-aware. The translations of code snippets are similar to those descriptions in queries, in which they share a range of words. We hence design a shared word mapping mechanism using one single vocabulary for generating embeddings for both translations and queries, substantially reducing the semantic discrepancy and improving the overall performance (see results in Section 5.2.2).

In summary, we make the following contributions.

- We propose a context-aware code translation technique that transforms code snippets into natural language descriptions with preserved semantics.
- We introduce a shared word mapping mechanism, which bridges the discrepancy of embedding representations from code snippets and queries.
- We implement a code search prototype called TranCS. We evaluate it on the CodeSearchNet corpus [25] with 1,000 queries. Experimental results show that TranCS improves the top-1 hit rate of code search by 67.16% to 102.90% compared to state-of-the-art techniques. In addition, TranCS achieves `MRR` of 0.651, outperforming DeepCS [18] and MMAN [62] by 66.50% and 49.31%, respectively. The source code of TranCS and all the data used in this paper are released and can be downloaded from the website [58].

2 BACKGROUND

2.1 Machine Instruction

Since the context-aware code translation technique we propose is performed at the machine instruction level, we first introduce the background about machine instructions.

A program runs by executing a sequence of machine instructions [11]. A machine instruction consists of an opcode specifying the operation to be performed, followed by zero or more operands embodying values to be operated upon [33, 45]. For example, in Java Virtual Machine, `istore_2` is a machine instruction where `istore` is an opcode whose operation is "store `int` into local variable", and `2` is an operand that represents the index of the local variable. Machine instructions have been widely used in software engineering activities, such as malware detection [3, 11, 45], API recommendation [47], code clone detection [60], program repair [15], and binary code search [65]. Machine instructions are generated by disassembling the binary files, such as the `.class` file in Java. Therefore, it is also called bytecode [15, 47, 57] or bytecode mnemonic opcode [60] in some of the works mentioned above. For ease of understanding, the terminology "instruction" is used uniformly in this paper.

2.2 Deep Learning-based Code Search

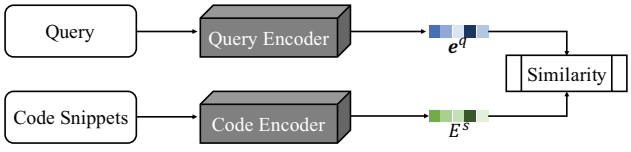


Figure 2: A General Framework of DL-based CS techniques

As shown in Figure 2, we can observe that deep learning (DL)-based CS techniques usually consist of three components, a query encoder, a code encoder, and a similarity measurement component. The query encoder is an embedding network that can encode the query q given by the developer into a d-dimensional embedding representation $\boldsymbol{e}^q \in \mathbb{R}^d$. To train such a query encoder, existing DL-based CS techniques have tried various neural network architectures, such as RNN [18], LSTM [62], and CNN [64]. In DL-based CS studies, it is a common practice to use code comments as queries during the training phase of the encoder [18, 56, 62]. Code comments are natural language descriptions used to explain what the

code snippets want to do [24]. For example, the first line of Figure 3(a) is a comment for the code snippet s_a. Therefore, we do not strictly distinguish the meaning of the two terms *comment* and *query*, and use the term *comment* during encoder training, and *query* at other times. The code encoder is also an embedding network that can encode n code snippets in the code corpus S into corresponding embedding representations $\boldsymbol{E}^S \in \mathbb{R}^{n \times d}$. In existing DL-based CS techniques, the code encoder is usually much more complicated than the query encoder. For example, the code encoder of MMAN [62] consists of three sub-encoders that are built on the LSTM [23], Tree-LSTM [59], and GGNN [32] architectures with the goal of encoding different features of the code snippet, e.g., tokens, ASTs, and CFGs. The similarity measurement component is used to measure the cosine similarity between $\boldsymbol{e}^q$ and each $\boldsymbol{e}^s \in \boldsymbol{E}^s$. The target of DL-based CS techniques is to rank all code snippets in S by the cosine similarity [18]. The higher the similarity, the higher relevance of the code snippet to the given query.

3 MOTIVATION

In this section, we study the limitations of commonly used representations of code snippets as well as the representation discrepancy between code snippets and comments in existing works [18, 62].

```
1  // calculate the sum of an int array
2  public int calArraySum(int[] array) {
3      int sum = 0;
4      int i = 0;
5      for (; i < array.length; i++) {
6          sum = sum + array[i];
7      }
8      return sum;
9  }
```

(a) Code Snippet s_a

```
1  // calculate the  sum of an int array
2  public int calArraySum(int[] array) {
3      int result = 0;
4      int index = 0;
5      while(index < array.length) {
6          result = result + array[index];
7          index++;
8      }
9  return result;}
```

(b) Code Snippet s_b

Figure 3: Code Snippets

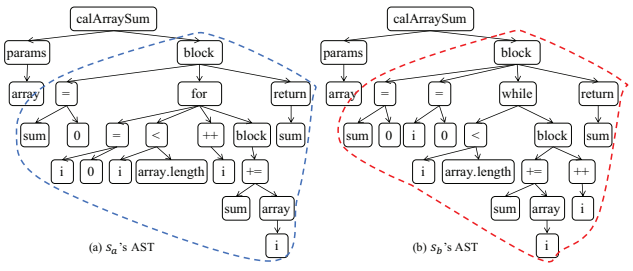


Figure 4: Abstract Syntactic Trees

Figure 3 shows two code snippets for calculating the sum of a given `int` array. Figure 3(a) uses a `for` statement to loop over all the elements in the array (line 5) and add their values to variable `sum` (line 6). Figure 3(b) employs a `while` statement for the same task (lines 5-8). Semantically, the two code snippets have the exact same meaning. In Figure 4, we show the abstract syntax trees (ASTs) for the above two code snippets s_a (left figure) and s_b (right figure),

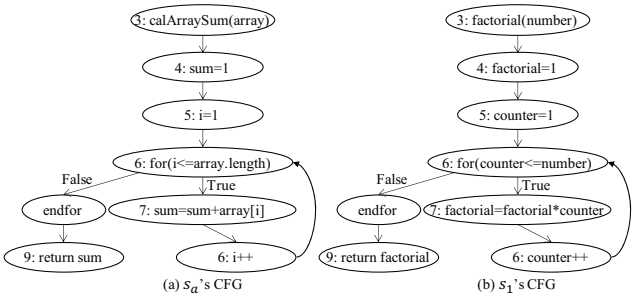


Figure 5: Control Flow Graphs

respectively. Observe that the sub-trees circled in dotted lines are different for the two code snippets. Such representations cause the inconsistency of code semantics, leading to inferior results in code search as we will show in Section 5.2.1. Control flow graph (CFG) is also commonly used for representing code snippets. Figure 5 depicts the CFGs for the two code snippets s_a and s_1 (see Figure 1(b) in Section 1). The task of s_1 is to calculate the factorial of a given number, while s_a is to calculate the sum of a given array. The two code snippets have completely different goals. However, the CFGs shown in Figure 5 have the same graph structure, which cannot differentiate the semantic difference between the two code snippets. This example delineates the insufficiency of utilizing CFGs for representing code semantics. Our experimental results in Section 5.2.1 show that a state-of-the-art technique MMAN [62] leveraging ASTs and CFGs has a limited performance.

Existing techniques leverage deep learning models (i.e., the encoders introduced in Section 2.2) for code search, where code snippets and comments need to be transformed into numerical forms in order to train those models and produce desired outputs. A common way is to build vocabularies for code snippets and comments, and construct corresponding numerical representations (e.g., word embeddings). A word mapping function is a dictionary with the key of a token in code snippets or a word in comments (from vocabularies) and the value of a fixed-length real-valued vector. DeepCS [18] builds four mapping functions for method names (MN), API sequences (APIs), tokens, and comments, separately. MMAN [62] utilizes four different mapping functions for tokens, ASTs, CFGs, and comments, respectively. The embeddings in these mapping functions are randomly initialized and learned during the training process of the encoder. Such a learning procedure introduces discrepant embedding representations for a same key (e.g., a code token). For instance, ASTs are composed of code tokens, which share a portion of same keys with the token vocabulary. Token names can also appear in comments. For example, more than 50% of keys appear in both code snippets and comments vocabularies used by DeepCS and MMAN. Inconsistent embeddings for same words/tokens can lead to unsuitable matches between code snippets and comments, causing poor performance of code search (see Section 5.2.2).

Our solution. We propose a novel code search technique, called TranCS, that better preserves the semantics of code snippets and bridges the discrepancy between code snippets and comments. Different from existing techniques that leverage ASTs and CFGs, we

```
0: push int constant 0.
1: store int 0 into local variable sum/result.
2: push int constant 0.
3: store int 0 into local variable i/index.
4: load int value from local variable i/index.
5: load reference array/array from local variable array/array.
6: get length of array array/array.
7: if and only if int value is greater or equal to int length then go to 22.
10: load int value_1 from local variable sum/result.
11: load reference array/array from local variable array/array.
12: load int value_2 from local variable i/index.
13: load int value_3 from array/array[value_2].
14: int result is int value_1 add int value_3; push result into value_4.
15: store int value_4 into local variable sum/result.
16: increment local variable i/index by constant 1.
19: goto 4.
22: load int value_5 from local variable sum/result.
23: return int value_5 from method.
```

Figure 6: Code Translations of s_a and s_b

directly translate code snippets into natural language sentences. Specifically, we utilize a standard program compiler and a disassembler to generate the instruction sequence of a code snippet. Such a sequence, however, lacks the context information such as local variables, data dependency, etc. We propose to simulate the execution of instructions to collect those desired contexts. A set of pre-defined translation rules are then used to translate the instruction sequence and contexts into natural language sentences. Details can be found in Section 4. Figure 6 showcases the translations of the two code snippets s_a and s_b by TranCS. The different colors denote different variable names used in s_a (blue) and s_b (red). The numbers/words in bold (e.g., **value** and **22**) denote the data and control dependencies among instructions. Observe that the translations of s_a and s_b are the same except for local variable names. The overall semantics described by the sentences in Figure 6 are the same. The translations are similar to those descriptions in comments, in which they share a range of words. We hence design a shared word mapping function using one single vocabulary for generating embeddings for both code snippets and comments, substantially reducing the semantic discrepancy and improving the overall performance (see results in Section 5.2.2).

4 METHODOLOGY

4.1 Overview

Figure 7 illustrates the overview of our TranCS. The top part shows the training procedure of TranCS and the bottom part shows the usage of TranCS for a given query. During the training procedure of TranCS, two types of input data are leveraged: comments and code snippets. The comments in Figure 7 are natural language descriptions that appear above the code snippet (e.g., Javadoc comments), not in the code body. These comments are input to TranCS in pairs with the corresponding code snippets to train CEncoder and TEncoder. For comments, TranCS transforms them into vector representations $\boldsymbol{V}^C$ using a shared word mapping function. For code snippets, they are different from natural language expressions such as comments. In this paper, we aim to build a homogeneous representation

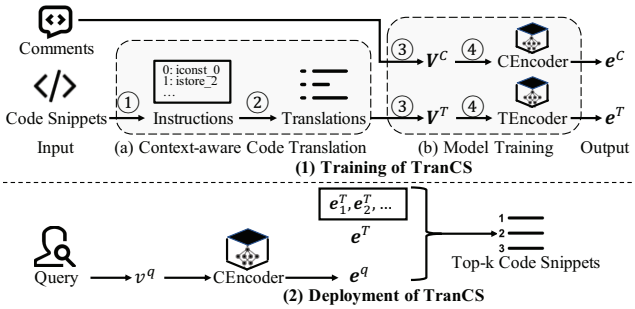


Figure 7: Framework of TranCS

between comments and code snippets, which can better capture the shared semantic information of these two types. Specifically, we propose a context-aware code translation, which translates code snippets into natural language descriptions as shown in the dotted box (details are discussed in Section 4.2). The natural language descriptions translated from code snippets are also transformed into vector representations $\boldsymbol{V}^T$ using the same shared word mapping function. TranCS leverages the two vector representations $\boldsymbol{V}^C$ and $\boldsymbol{V}^T$ for building two encoders (i.e., CEncoder and TEncoder) that generate embeddings with preserved semantics for both comments and code snippets. CEncoder takes in the comment vector representations $\boldsymbol{V}^C$ and produces concise embedding representations $\boldsymbol{e}^C$ that preserves semantic information from the comments. TEncoder generates embedding representations $\boldsymbol{e}^T$ for code snippets. Details of training these two encoders are elaborated in Section 4.3. When TranCS is deployed for usage, it takes in a query from the developer and passes it to CEncoder, which produces an embedding $\boldsymbol{e}^q$ for the query. TranCS then compares the query embedding $\boldsymbol{e}^q$ with those code embeddings $\boldsymbol{e}^T$ from the training set. A top-k selection method is leveraged for providing code snippets to the developer, which are semantically similar to the query.

4.2 Context-aware Code Translation

The goal of context-aware code translation is to translate code snippets into natural language descriptions according to the predefined translation rules. As shown in the dotted box of Figure 7, this phase consists of two steps. In step ①, given code snippets, TranCS utilizes a standard compiler and disassembler to generate their instruction sequences. In step ②, TranCS applies the predefined translation rules to translate the instruction sequences into natural language descriptions. We discuss the two steps in detail in the following sections.

4.2.1 Instruction Generation. In this step, TranCS takes in code snippets and produces their instruction sequences. In practice, for a given code snippet, TranCS first utilizes a standard program compiler and disassembler to generate the disassembly representation of the code snippet. For example, TranCS integrates `javac` version 1.8.0_144 (a compiler) and `javap` version 1.8.0_144 (a disassembler) to generate the disassembly representations for code snippets written in the Java programming language. For the code snippets that can not be compiled, the main reason is due to the lack of class/method definitions around them. We use JCoffee [20] to make

them compilable by adding class/method definitions around them to complement the missing pieces. Then, TranCS parses the disassembly representation and extracts the instruction sequence. For example, Figure 8(a) shows an instruction sequence, which is generated by inputting the code snippet shown in Figure 3(a) into TranCS.

0: iconst_0	0: push int constant [pc].
1: istore_2	1: store int [ps] into local variable [pv].
2: iconst_0	2: push int constant [pc].
3: istore_3	3: store int [ps] into local variable [pv].
4: iload_3	4: load int [ps] from local variable [pv].
5: aload_1	5: load reference [ps] from local variable [pv].
6: arraylength	6: get length of array [ps].
7: if_icmpge 22	7: if int [ps] is greater or equal to int [ps] then go to [pi].
10: iload_2	10: load int [ps] from local variable [pv].
11: aload_1	11: load reference [ps] from local variable [pv].
12: iload_3	12: load int [ps] from local variable [pv].
13: iaload	13: load int [ps] from array [ps].
14: iadd	14: int result is int [ps] add int [ps]; push int result.
15: istore_2	15: store int [ps] into local variable [pv].
16: iinc 3, 1	16: increment local variable [pv] by constant [pc].
19: goto 4	19: goto [pi].
22: iload_2	22: load int [ps] from local variable [pv].
23: ireturn	23: return int [ps] from method.

(a) Instruction Sequence (b) Translation Rules

Figure 8: An Example of Instruction Sequence and Translation Rules. [pc] and [pv] indicate filling in a constant and variable, respectively. [ps] indicates filling a value popped from the operand stack, while [pi] indicates filling in an instruction index.

In addition to the instruction sequence, TranCS also extracts the local variable table from the disassembly representation, which will be used in the subsequent instruction translation process. For example, Listing 1 shows an example of a local variable table (LocalVariableTable) that presents the local variables involved in the code snippet in detail, and is generated along with the instruction sequence in Figure 8(a). Details about the usage of local variables are introduced in Section 4.2.2.

```
LocalVariableTable:
  Start  Length  Slot  Name   Signature
      0      24     0  this   LCalArraySum;
      0      24     1 array   [I
      2      22     2   sum   I
      4      20     3     i   I
```

Listing 1: An Example of Local Variable Table

4.2.2 Instruction Translation. In this step, TranCS takes in instruction sequences and produces their natural language descriptions. In this section, we first introduce the translation rules used in TranCS, then introduce the instruction context, and finally present how TranCS implements context-aware instruction translation.

Translation Rules (TR). TR used in TranCS is manually constructed based on the instruction specification. In practice, to construct TR, we collected all operations and descriptions of instructions from the machine instruction specification, such as Java Virtual Machine Specification [33]. An operation is a short natural language description of an instruction. For example, the instruction `istore`'s operation is:

"store `int` into local variable."

From this operation, we can know the behavior of `istore` is to store an `int` value into a local variable. A description is a long natural language description of an instruction, which details the interaction of the instruction on the local variables and operand stack. For example, `istore`'s description is:

"The *index* is an unsigned byte that must be an index into the local variable array of the current frame. The *value* on the top of the operand stack must be of type `int`. It is popped from the operand stack, and the value of the local variable at *index* is set to value."

From this description, we can know that `istore` first pops an `int` value from the operand stack and then stores the value into the *index*-th position of the local variable array. If we only use the operation as the translation of the instruction, the translation will be inaccurate due to the loss of some important context. If we only use the description as the translation of instructions, the translation will be too long. However, research in the field of natural language processing (NLP) reminds us that capturing the semantics of long texts is more difficult than short texts [2, 61]. Based on the above, we strive to make the instruction translation short and relatively accurate. Therefore, we use the operation as the basis, combing the context specified in the description, to manually collate a translation for each instruction. Such a translation delicately balances shortness and accuracy. For example, the translation we collate for the instruction `istore` as follows:

"store `int` [ps] into local variable [pv]."

where [ps] and [pv] denote placeholders that specifies the position where the context will be filled, and details about instruction context are discussed in Section **Context-aware Instruction Translation**. For example, Figure 8(b) shows the result of TranCS using TR to translate the instruction sequence in Figure 8(a).

Instruction Context. The context of an instruction consists of constants, local variables, and data and control dependencies with other instructions. Constants and local variables are directly determined by operands. As shown in Figure 8(a), an opcode is followed by zero or more operands. An operand can be a constant, or an index of a local variable, or an index of an instruction. For example, in Figure 8(a), the operand 0 following the opcode `iconst` represents a constant, while the operand 2 following the opcode `istore` represents the index of the local variable *sum* shown in Listing 1; Control dependencies between instructions are explicitly passed through the indices of the instruction. The indices are also directly specified by operands. For example, the operand 22 following the opcode `if_icmpge` represents the index of the instruction `iload_2` at line 22 in Figure 8(a). Data dependencies between instructions are implicitly passed through the operand stack. As described in Section **Translation Rules (TR)**, with the guidance of the description, we can know how each instruction interacts with the operand stack, such as popping or pushing data. If the instruction i_a pops (i.e., uses) the data that is pushed onto the operand stack by the instruction i_b, then we say that i_a is data dependent on i_b. For example, Figure 9(a) shows the changes of the operand stack as the opcode sequence in Figure 8 interacts with the operand stack. The values in the operand stack are the carriers that reflect data

dependencies between instructions. Figure 9(b) shows the data and control dependencies between the instructions in Figure 8(a). In this figure, nodes represent instructions; the labels of nodes are instructions' indices; the solid and dashed edges represent data and control dependencies, respectively.

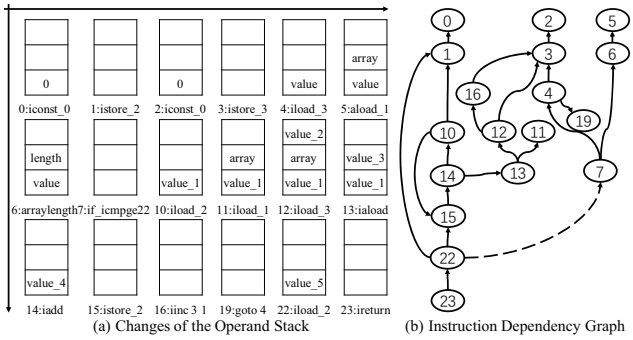


Figure 9: An Example of the Changes of the Operand Stack and Instruction Dependency Graph

Context-aware Instruction Translation. The basic idea of context-aware instruction translation is to simulate the execution of instructions by statically traversing the instruction sequence from top to down. In the traversal process, we collect the context of each instruction, which will be used to update the TR-based translations of the current or other related instructions.

In the actual execution of instructions, a frame is created when the corresponding code snippet is invoked [34]. A frame contains a local variable array and a last-in-first-out stack (i.e., operand stack). The sizes of the local variable array and the operand stack are determined at compile-time. The local variable array stores all local variables used in the instructions. For example, the local variables shown in Listing 1 are used in the instruction sequence shown in Figure 8(a). The indices of the local variable array corresponds to that in LocalVariableTable shown in Listing 1, where the 'Slot' column presents indices of the local variables. The names and indices of local variables are determined at compile-time, but their values are dynamically updated with the execution of the instructions. The values in the operand stack are also dynamically updated with the execution of the instructions. As mentioned earlier, the context of an instruction includes constants, local variables, data and control dependencies. Among them, constants, local variables and control dependencies are closely related to instructions' operands. They can be easily determined by the operands (for constants) or by retrieving the instruction sequence (for control dependencies) using the index specified by the operand. However, determining the values of local variables is a challenging task because they are dynamically updated with the execution of the instruction. Analogously, the determination of data dependencies is a challenging task because they are implicitly passed through the operand stack. The values in operand stack are also dynamically updated with the execution of the instruction. Therefore, we need to know in advance how the instruction interacts with the local variable array (e.g., setting value) or the operand stack (e.g., popping or pushing data). In practice, we obtain such information from the description of each instruction. The description of each instruction has been introduced when we introduced the translation rules earlier. With the guidance of the description, we divide the instructions into the following four categories according to whether they interact with the local variable array or the operand stack.

Category 1, expressed as $\mathbb{I}^S$. In $\mathbb{I}^S$, the instruction only interacts with the operand stack. $\mathbb{I}^S$ can be subdivided into the following three types:

- $\mathbb{I}^{PU}$. In this type, the interaction is to push the operand onto the operand stack.
- $\mathbb{I}^{PO}$. In this type, the interaction is to pop values from the operand stack.
- $\mathbb{I}^{POU}$. In this type, the interaction is composed of popping values from the operand stack, performing the operation, and pushing the result of the operation to the operand stack.

Category 2, $\mathbb{I}^V$. In $\mathbb{I}^V$, the instruction only interacts with the local variable array. The interaction is to load the value from the local variable array, or store the new value into it. This type of instruction does not interact with the operand stack. For example, the instruction `iinc 3, 1` only interacts with the local variable specified by the first operand, not with the operand stack.

Category 3, $\mathbb{I}^{SV}$. In $\mathbb{I}^{SV}$, the instruction interacts with the operand stack as well as the local variable array. For example, the instruction `istore_2` first loads the integer value from the operand stack, and then stores the value into a local variable.

Category 4, $\mathbb{I}^O$. In $\mathbb{I}^O$, the instruction neither interacts with the operand stack nor with the local variable array, such as the instruction `goto` and `nop`. Table 1 shows the categories of instructions.

Based on the above classification, TranCS uses Algorithm 1 to perform context-aware instruction translation. TranCS takes an instruction sequence (I), translation rules (TR), a local variable array (V), and the depth of the operand stack (d) as inputs. TR, V and d have been introduced earlier. TranCS first initializes an stack with a depth of d to store intermediate results produced during traversing I (line 1). TranCS then traverses I from top to down (lines 2 – 33). For each $i \in I$, TranCS first generates its translation t based on TR (line 3). Then, TranCS extracts the operands from i (line 4). The operands are used to update S and t in subsequent processes. TranCS determines i's category according to the predefined categories shown in Table 1. According to i's category, TranCS uses different processes to update S and t (line 5 – 31). For example, Figure 9(a) shows an example of the changes of the operand stack when TranCS traverses the instruction sequence shown in Figure 8(a) from top to down. After traversing all the instructions in I, the algorithm finishes and outputs I's translations T. For example, Figure 6 shows the translation generated by TranCS for the instruction sequence shown in Figure 8(a).

4.3 Model Training

The goal of model training is to train two encoders, which will be deployed to support code search service. This phase consists of two steps as shown in Figure 7. In step ③, given translations and comments, TranCS transforms them into vector representations $\boldsymbol{V}^C$ and $\boldsymbol{V}^T$ using a shared word mapping function. In step ④, TranCS leverages $\boldsymbol{V}^C$ and $\boldsymbol{V}^T$ to train CEncoder and TEncoder.

Table 1: The Category of Instructions

Category	$\mathbb{I}^S$			$\mathbb{I}^V$	$\mathbb{I}^{SV}$	$\mathbb{I}^O$
Type	$\mathbb{I}^{PU}$	$\mathbb{I}^{PO}$	$\mathbb{I}^{POU}$			
Instructions	aconst_null, anewarray, iconst, fconst, bipush, dconst_<d>, fconst_<f>, iconst_<i>, jsr, jsr_w, lconst_<l>, ldc, ldc_w, ldc2_w, new, sipush	areturn, if_icmpge, ireturn, athrow, dreturn, freturn, if_acmp<cond>, if_icmp<cond>, if<cond>, ifnonnull, ifnull, invokedynamic, invokeinterface, invokespecial, invokestatic, invokevirtual, ireturn, ishl, ishr, lookupswitch, lreturn, monitorexit, pop, pop2, putfield, putstatic, tableswitch	aaload, arraylength, baload, caload, d2f, d2i, d2l, dadd, daload, dcmp<op>, ddiv, dmul, dneg, drem, dsub, dup, dup_x1, dup_x2, dup2, dup2_x1, dup2_x2, f2d, f2i, f2l, fadd, faload, fcmp<op>, fdiv, fmul, fneg, frem, fsub, getfield, getstatic, i2b, i2c, i2d, i2f, i2l, i2s, iadd, iaload, iand, idiv, imul, ineg, instanceof, ior, irem, isub, iushr, ixor, l2d, l2f, l2i, ladd, laload, land, lcmp, ldiv, lmul, lneg, lor, lrem, lshl, lshr, lsub, lushr, multianewarray, lxor, newarray, saload, swap	iinc, wide	aastore, aload, aload_<n>, astore, astore_<n>, bastore, castore, dastore, dload, dload_<n>, dstore, dstore_<n>, fastore, fload, fload_<n>, fstore, fstore_<n>, iastore, iload, iload_<n>, istore, istore_<n>, lastore, lload, lload_<n>, lstore, lstore_<n>, sastore	goto, checkcast, goto_w, nop, ret, return

Algorithm 1 Context-aware Instruction Translation

Input: An instruction sequence, I; Translation Rules, TR;
A local variable array, V; The depth of the operand stack, d.
Output: Instruction Translation, T;

$S \leftarrow$ initialize an empty operand stack with a depth of d.
for each i in I **do**
  $t \leftarrow$ generate the TR-based translation of i based on TR;
  $operands \leftarrow$ extract the operands from i;
  if $i \in \mathbb{I}^{PU}$ **then**
    $S \leftarrow$ push $operands$ onto S;
    $t \leftarrow$ replace [pc] in t with $operands$;
  end if
  if $i \in \mathbb{I}^{PO}$ **then**
    $values \leftarrow$ pop values from S by $operands$;
    $t \leftarrow$ replace [ps] in t with $values$;
  end if
  if $i \in \mathbb{I}^{POU}$ **then**
    $values \leftarrow$ pop values from S by $operands$;
    $t \leftarrow$ replace [ps] in t with $values$;
    $new\_value \leftarrow$ do operation;
    $S \leftarrow$ push new_value onto S;
  end if
  if $i \in \mathbb{I}^{V}$ **then**
    $variable \leftarrow$ get variable from V by $operands$;
    $t \leftarrow$ replace [pv] in t with $variable$;
  end if
  if $i \in \mathbb{I}^{SV}$ **then**
    $values \leftarrow$ pop values from S by $operands$;
    $t \leftarrow$ replace [ps] in t with $values$;
    $variable \leftarrow$ get variable from V by $operands$;
    $t \leftarrow$ replace [pv] in t with $variable$;
  end if
  if t contains [pi] **then**
    $t \leftarrow$ replace [pi] in t with $operands$;
  end if
  $T \leftarrow T \cup \{t\}$
end for
output T;

4.3.1 Shared Word Mapping. In TranCS, both translations and comments are natural language sentences. Sentence embedding is generated based on word embedding [43, 50]. Word embedding techniques can map words into fixed-length vectors (i.e., embeddings) so that similar words are close to each other in the vector space [42, 43].

A word embedding technique can be considered a word mapping function ψ, which can map a word w_i into a vector representation $\boldsymbol{w_i}$, i.e., $\boldsymbol{w_i} = \psi(w_i)$. As aforementioned, both translations and comments are natural language sentences, so we design a shared word mapping function. To implement such a ψ, we build a shared vocabulary that includes top-n frequently appeared words in translations and comments. We further transform the vector representations of the words into an embedding matrix $E \in \mathbb{R}^{n \times m}$, where n is the size of the vocabulary, m is the dimension of word embedding. The embedding matrix $E = (\psi(\boldsymbol{w}_1), ..., \psi(\boldsymbol{w}_i))^T$ is initialized randomly and learned in the training process along with the two encoders. Based on this embedding matrix, TranCS can transforms translations and comments into the vector representations $\boldsymbol{V}^C$ and $\boldsymbol{V}^T$. A simple way of sentence vector representations is to view it as a bag of words and add up all its word vector representations [30].

4.3.2 Encoder Training. In this section, we first introduce the architecture of CEncoder and TEncoder, then present how to jointly train the two encoders.

Encoder Architecture. As described in Section4.3.1, in TranCS both translations and comments are natural language sentences. Therefore, we can use the same sequence embedding network to design comment encoder (CEncoder) and translation encoder (TEncoder) instead of designing different embedding networks for them as the previous DL-based CS techniques, such as DeepCS [18] and MMAN [62]. In practice, TranCS applies the LSTM architecture to design CEncoder and TEncoder. Consider a translation/comment sentence $s = w_1, \cdots, w_{N^s}$ comprising a sequence of N^s words, TranCS first uses the shared word mapping function to produce vector representations $\boldsymbol{v}^s$. Then, TranCS passes $\boldsymbol{v}^s$ to the encoder

(i.e., CEncoder or TEncoder) that generates embeddings $\boldsymbol{e}^s$. The hidden state $\boldsymbol{h}_i^s$ of the i-th word in s is calculated as follows:

$$\boldsymbol{h}_i^s = LSTM(\boldsymbol{h}_{i-1}^s, \boldsymbol{w}_i) \tag{1}$$

where $\boldsymbol{w}_i$ represents the vector of the word w_i and comes from the embedding matrix E.

In addition, TranCS uses attention mechanism proposed by Bahdanau et al. [1] to alleviate the long-dependency problem in long text sequences [2]. The attention weight for each word w_i is calculated as follows:

$$\alpha_i^s = \frac{exp(f(\boldsymbol{h}_i^s) \cdot \boldsymbol{u}^s)}{\sum_{j=1}^{N^s} exp(f(\boldsymbol{h}_j^s) \cdot \boldsymbol{u}^s)} \tag{2}$$

where $f(\cdot)$ denotes a linear layer; $\boldsymbol{u}^s$ denotes the context vector which is a high level representation of all words in s; and $\cdot$ denotes the inner project of $\boldsymbol{h}_i^s$ and $\boldsymbol{u}^s$. The context vector $\boldsymbol{u}^s$ is randomly initialized and jointly learned during training. Then, s's final embedding representation $\boldsymbol{e}^s$ can be calculated as follows:

$$\boldsymbol{e}^s = \sum_{j=1}^{N^s} \alpha_i^s \cdot \boldsymbol{h}_i^s \tag{3}$$

Joint Training. Now we present how to jointly train the two encoders (i.e., CEncoder and TEncoder) of TranCS to transform both translations and comments into a unified vector space with a similarity coordination. We follow a widely adopted assumption that if a translation and a comment have similar semantics, their embedding representations should be close to each other [18, 56, 62]. In other words, given a code snippet s whose translation is t and a comment c, we want it to predict a high similarity between t and c if c is a correct comment of s, and a little similarity otherwise.

In practice, we first translate all code snippets into translations. Then, we construct each training instance as a triple $\langle t, c^+, c^- \rangle$: for each translation t there is a positive comment c^+ (a ground-truth comment of s) and a negative comment c^- (an incorrect comment of s). The incorrect comment c^- is selected randomly from the pool of all correct comments. When trained on the set of $\langle t, c^+, c^- \rangle$ triples, TranCS predicts the cosine similarities of both $\langle t, c^+ \rangle$ and $\langle t, c^- \rangle$ pairs and minimizes the ranking loss [10, 14]:

$$\mathcal{L}(\theta) = \sum_{\langle \boldsymbol{t}, \boldsymbol{c}^+, \boldsymbol{c}^- \rangle \in G} max(0, \beta - cos(\boldsymbol{t}, \boldsymbol{c}^+) + cos(\boldsymbol{t}, \boldsymbol{c}^-)) \tag{4}$$

where θ denotes the model parameters; G denotes the training dataset; β is a small and fixed margin constraint; $\boldsymbol{t}$, $\boldsymbol{c}^+$ and $\boldsymbol{c}^-$ are the embedded vectors of t, c^+ and c^-, respectively. Intuitively, the ranking loss encourages the cosine similarity between a translation and its correct comment to go up, and the cosine similarities between a translation and incorrect comments to go down.

4.4 Deployment of TranCS

After the two encoders (i.e., CEncoder and TEncoder) are trained, we can deploy TranCS online for code search service. Figure 7(2) shows the deployment of TranCS. For a search query q given by the developer, TranCS first uses the shared word mapping function to transform it into vector representation $\boldsymbol{v}^q$. TranCS further passes $\boldsymbol{v}^q$ into CEncoder to generate the embedding $\boldsymbol{e}^q$. Then, TranCS measures the similarity between $\boldsymbol{e}^q$ and each $\boldsymbol{e}^t \in \boldsymbol{e}^T$. The similarity is calculated as follows:

$$sim(q, t) = cos(\boldsymbol{e}^q, \boldsymbol{e}^t) = \frac{\boldsymbol{e}^q \cdot \boldsymbol{e}^t}{\|\boldsymbol{e}^q\| \, \|\boldsymbol{e}^t\|} \tag{5}$$

TranCS ranks all $\boldsymbol{T}$ by their similarities with q. The higher the similarity, the higher relevance of the code snippet to q. Finally, TranCS outputs the code snippets corresponding to the top-k translations to the developer.

5 EVALUATION AND ANALYSIS

We conduct experiments to answer the following questions:

RQ1. What is the effectiveness of TranCS when compared with state-of-the-art techniques?

RQ2. What is the contribution of key components in TranCS, i.e., context-aware code translation and shared word mapping?

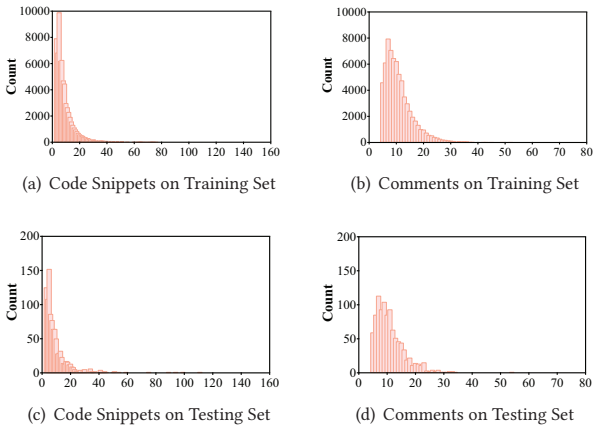
RQ3. What is the robustness of TranCS when varying the query length and code length?

5.1 Experimental Setup

5.1.1 Dataset. We evaluate the performance of our TranCS on a corpus of Java code snippets, collected from the public CodeSearchNet corpus [9]. Actually, we have considered the dataset released by baselines (i.e., DeepCS [18] and MMAN [62]). However, the dataset of DeepCS only contains the cleaned Java code snippets without the raw data, unable to generate the CFG for MMAN. And the dataset of MMAN is not publicly accessible.

We randomly shuffle the dataset and split it into two parts, i.e., 69,324 samples for training and 1,000 samples for testing. It is worth mentioning a difference between our data processing and the one in [18]. In [18], the proposed approach is verified on another isolated dataset to avoid the bias. Since the evaluation dataset does not have the ground truth, they manually labelled the searched results. As possible subjective bias exists in manual evaluation [7, 62], in this paper, we also adopt the automatic evaluation. Figure 10(a) and (b) show the length distributions of code snippets and comments on the training set. For a code snippet, its length refers to the number of lines of the code snippet. For a comment, its length refers to the number of words in the comment. From Figure 10(a), we can observe that the lines of most code snippets are located between 20 to 40. This was also observed in the quote in [38] "Functions should hardly ever be 20 lines long". From Figure 10(b), it is noticed that almost all comments are less than 20 in length. This also confirms the challenge of capturing the correlation between short text with its corresponding code snippet. Figure 10(c) and (d) show the length distributions of code snippets and comments on testing data. We can observe that, despite shuffling randomly, the distributions of data sizes (i.e., lengths) in the two data sets are consistent, so we can conclude that the testing set is representative.

5.1.2 Evaluation Metrics. In the evaluation, we consider the comment of the code snippet as the query, and the code snippet itself as the ground-truth result of code search, which is similar to [21, 56, 62] but different from [7, 18]. During the testing time, we treat each comment in the 1,000 testing samples as a query, the code snippet corresponding to the query as the correct result, and



(a) Code Snippets on Training Set (b) Comments on Training Set

(c) Code Snippets on Testing Set (d) Comments on Testing Set

Figure 10: Length Distributions

the other 999 code snippets as distractor results. We adopt two automatic evaluation metrics that are widely used in code search studies [7, 18, 21, 56, 62] to measure the performance of TranCS, i.e., success rate at k (SuccessRate@k) and mean reciprocal rank (MRR).

SuccessRate@k measures the percentage of queries for which the correct result exists in the top k ranked results [28, 62], which is computed as follows:

$$\text{SuccessRate@k} = \frac{1}{|Q|}\sum_{i=1}^{|Q|}\delta(FRank_{Q_i} \leq k) \tag{6}$$

where Q denotes a set of queries and $|Q|$ is the size of Q; $\delta(\cdot)$ denotes a function which returns 1 if the input is true and returns 0 otherwise; $FRank_{Q_i}$ refers to the rank position of the correct result for the i-th query in Q. SuccessRate@k is important because a better CS technique should allow developers to discover the expected code snippets by inspecting fewer returned results. The higher the SuccessRate@k value, the better the code search performance.

MRR is the average of the reciprocal ranks of results of a set of queries Q [56, 62]. The reciprocal rank of a query is the inverse of the rank of the correct result. MRR is computed as follows:

$$\text{MRR} = \frac{1}{|Q|}\sum_{i=1}^{|Q|}\frac{1}{FRank_{Q_i}} \tag{7}$$

The higher the MRR value, the better the code search performance.

Meanwhile, as developers prefer to find the expected code snippets with short inspection, we only test SuccessRate@k and MRR on the top-10 (that is, the maximum value of k is 10) ranked list following DeepCS [18] and MMAN [62]. In other words, when the rank of Q_i is out of 10, then $1/FRank_{Q_i}$ is set to 0.

5.1.3 Baselines. In this paper, we compare the following baselines:

- **DeepCS** [18]. DeepCS is one of the representative DL-based CS techniques. DeepCS uses two kinds of model architecture to design the code encoder to embed three aspects of the code snippet, i.e., two RNNs for method names and API sequences, and a multi-layer perceptron (MLP) for tokens. Its query encoder also uses RNN architecture.
- **MMAN** [62]. MMAN is one of the state-of-the-art DL-based CS techniques. MMAN uses multiple kinds of model architectures to design the code encoder to embed multiple aspects of the code snippet, i.e., one LSTM for Token, a Tree-LSTM for AST, and a GGNN for CFG. Its query encode uses LSTM architecture.

5.1.4 Implementation Details. To train our model, we first shuffle the training data and set the mini-batch size to 32. The size of the vocabulary is 15,000. For each batch, the code snippet is padded with a special token $\langle PAD \rangle$ to the maximum length. We set the word embedding size to 512. For LSTM unit, we set the hidden size to 512. The margin β is set to 0.6. We update the parameters via AdamW optimizer [29] with the learning rate 0.0003. To prevent over-fitting, we use dropout with 0.1. In TranCS, the comment and the code snippet share the same embedding weights. All models are implemented using the PyTorch 1.7.1 framework with Python 3.8. All experiments are conducted on a server equipped with one Nvidia Tesla V100 GPU with 31 GB memory, running on Centos 7.7. All the models in this paper are trained for 200 epochs, and we select the best model based on the lowest validation loss.

5.2 Evaluation Results

In this section, we present and analyze the experimental results to answer the research questions.

5.2.1 ***RQ1:*** *Effectiveness of TranCS.* Table 2 shows the overall performance of TranCS and two baselines, measured in terms of SuccessRate@k and MRR. The columns SR@1, SR@5 and SR@10 show the results of the average SuccessRate@k over all queries when k is 1, 5 and 10, respectively. The column MRR shows the MRR values of the three techniques. From this table, we can observe that for SR@k, the improvements of TranCS to DeepCS are 102.90%, 45.80% and 32.48% when k is 1, 5, and 10, respectively. The improvements to MMAN are 67.16%, 35.94%, and 25.42%, respectively. For MRR, the improvements TranCS to DeepCS and MMAN are 66.50% and 49.31%, respectively. We can draw the conclusion that under all experimental settings, our TranCS consistently achieves higher performance in terms of both two metrics, which indicates better code search performance.

Table 2: Overall Performance of TranCS and Baselines

Tech	SR@1	SR@5	SR@10	MRR
DeepCS	0.276	0.524	0.622	0.391
MMAN	0.335	0.562	0.657	0.436
TranCS	**0.560**	**0.764**	**0.824**	**0.651**

The CodeSearchNet corpus also provides 99 realistic natural languages queries and expert annotations for likely results. Each query/result pair was labeled by a human expert, indicating the relevance of the result for the query. We also conduct experiments on 99 queries provided by the CodeSearchNet corpus for the Java

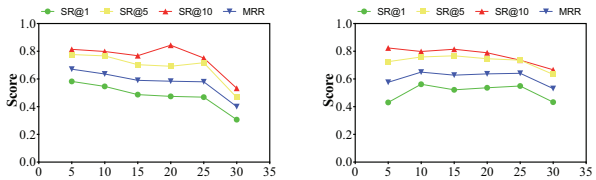
programming language. We use the same metric, normalized discounted cumulative gain (NDCG [55]), to evaluate baselines and TranCS. Our TranCS achieves NDCG of 0.223, outperforming DeepCS (**0.138**) and MMAN (0.173) by 62% and 30%, respectively.

Table 3: Contribution of Key Components in TranCS

Tech	SR@1	SR@5	SR@10	MRR
TokeCS	0.247	0.477	0.586	0.359
TranCS (CCT)	**0.352**	**0.569**	**0.664**	**0.455**
TokeCS (SWM)	0.264	0.483	0.592	0.370
DeepCS (SWM)	0.295	0.511	0.615	0.399
TranCS (CCT+SWM)	**0.560**	**0.764**	**0.824**	**0.651**

5.2.2 ***RQ2:*** *Contribution of Key Components.* We experimentally verified the effectiveness of two key components of TranCS i.e., context-aware code translation (CCT) and shared word mapping (SWM). In Table 3, TranCS(CCT) and TranCS(CCT+SWM) are two special versions of TranCS, among which the former uses two different word mapping functions to transform instruction translations and comments to vector representations, while the latter uses SWM. In other words, if it is only CCT, TranCS uses two vocabularies. In the case of CCT+SWM, TranCS uses a shared vocabulary. Moreover, numerous existing studies [53, 56, 68] including DeepCS [18] and MMAN [62] have shown that tokens of code snippets play a key role in code search tasks. Therefore, we assume that this is a scenario where the code snippet is not translated, and we directly pass the tokens of the code snippet into the model to train the code encoder. The effectiveness of the token-based CS technique (TokeCS) is shown in the second line of Table 3. To demonstrate the effectiveness of SWM, we also tried to apply SWM to TokeCS, DeepCS and MMAN. To apply SWM to TokeCS, we use a unified word mapping function to transform tokens and comments. In DeepCS, the author uses four word mapping functions to transform the MN, APIS, Token and comments into vector representations. To apply SWM to DeepCS, we first merge the four vocabularies into a shared vocabulary by extracting the union of them. Then, we use a unified word mapping function to transform MN, APIS, Token and comments. In MMAN, the author not only uses LSTM architecture to embed tokens, but also uses Tree-LSTM and GGNN to embed AST and CFG, while the three architectures cannot share a word mapping function. Therefore, SWM can not be applied to MMAN. The effectiveness of Toke(SWM), DeepCS(SWM) are shown in lines 4–5 of Table 3. From the lines 2–3 of Table 3, we can observe that for SR@k, the improvements of TranCS(CCT) to TokeCS are 42.51%, 19.29% and 13.31% when k is 1, 5, and 10, respectively. For MRR, the improvement to TokeCS is 26.74%. Therefore, we can conclude that CCT contributes to TranCS. For SR@k, the improvements of TranCS(CCT+SWM) to TranCS(CCT) are 59.09%, 34.27% and 24.10%. For MRR, the improvement of TranCS(CCT+SWM) to TranCS(CCT) is 43.08%. Therefore, we can conclude that SWM contributes to TranCS. Besides, we can also observe that SWM also has slight improvements to TokeCS and DeepCS. Therefore, we can draw the conclusion that SWM and CCT, which promote each other, improve the performance of TranCS jointly.

5.2.3 ***RQ3:*** *Robustness of TranCS.* To analyze the robustness of TranCS, we studied two parameters (i.e., code length and comment length) that may have an impact on the embedding representations of translations and comments. Figure 11 shows the performance of TranCS based on different evaluation metrics with varying parameters. From Figure 11, we can observe that in most cases, TranCS maintains a stable performance even though the code snippet length or comment length increases, which can be attributed to context-aware code translation and shared word mapping we proposed. When the length of the code snippet exceeds 20 (a common range described in Section 5.1.1), the performance of TranCS decreases as the length increases. It means that when the length of the code snippets or comments exceeds the common range, as the length continues to increase, it will be more difficult to capture their semantics. Overall, the results verify the robustness of our TranCS.



(a) Varying Code Snippet Lengths (b) Varying Comment Lengths

Figure 11: Robustness of TranCS

6 CASE STUDY

swap two elements in the list

(a) Query q

```
1 void swapElementInList(List<Integer> list, int i, int j) {
2     int element = list.get(i);
3     list.set(i, list.get(j));
4     list.set(j, element);
5 }
```

(b) Code Snippet s_1

```
1 void swapElementInList(List<Integer> list, int i, int j) {
2     Collections.swap(list, i, j);
3 }
```

(c) Code Snippet s_2

Figure 12: Example of Two Code Snippets Implementing the Same Functionality

This is a case to study the performance of TranCS in retrieving code with implantation difference. Figure 12(b) and (c) show two code snippets that implement the same functionality, i.e., swapping

two elements in the list. The first one (s_1) implements the functionality from scratch, and the second one (s_2) directly calls the external API *Collection.swap*(). We use TranCS to convert the two code snippets into corresponding translations, which are very different, meaning TranCS can effectively differentiate semantically similar code but differs in APIs used. This is because TranCS reserves API information (e.g., name, parameter) when generating code translation. For example, as shown in Figure 13, the translations produced by TranCS reserve the information of the API *Collection.swap*() invoked by s_2, including the parameters (e.g., **list**) and the method name **swap**.

```
0: load reference list from local variable list.
1: load int i from local variable i.
2: load int j from local variable j.
3: invoke class Collections static method swap.
4: return void from method.
```

Figure 13: Translations of the Code Snippet s_2

7 THREATS TO VALIDITY

The metrics used in this paper are `SuccessRate@k` and `MRR` for evaluating the effectiveness of TranCS and existing techniques. These are the same metrics adopted in MMAN [62]. We do not use another metric `Precision@k` that measures the percentage of relevant results in the top k returned results for each query [18]. This is due to the constraint that the relevant results need to be labelled manually, which is empirically less feasible and can introduce human biases. We hence focus on the two metrics `SuccessRate@k` and `MRR` in the paper.

TranCS is currently only evaluated on Java programs and may require modifications for extending to other programming languages. The core contribution of TranCS is the context-aware code translation technique. To realize the context-aware code translation, TranCS requires a set of translation rules, such as the operations and descriptions of instructions. In order to extend TranCS to other programming languages, corresponding translation rules need to be designed and provided. We plan to evaluate the performance of TranCS on these programming languages in future work.

8 RELATED WORK

Early CS techniques were based on IR technology, such as [4, 27, 40, 51, 54]. These techniques simply consider queries and code snippets as plain text and then use keyword matching. To alleviate the problem of keyword mismatch [8, 22] and noisy keywords [19], many query reformulation(QR)-based CS techniques [22, 31, 36, 37, 48, 52] have been proposed one after another. For example, the words from WordNet [44], or Stack Overflow [48] are used to expand user queries. However, QR-based CS techniques consider each word independently, while ignoring the context of the word. In addition, both IR-based and QR-based CS techniques only treat the code snippet as plain text, and cannot capture the deep semantics of the code snippet. To better capture the semantics of queries and code snippets, deep learning (DL)-based CS techniques [7, 13, 18, 25, 53, 56, 62, 66] have been proposed one after another. Gu et al. [18] first apply DL to the code search task. They first encode both the query and a set of code snippets into corresponding embeddings using MLP or RNN, and then rank the code snippets according to the cosine similarity of embeddings. Other DL-based CS techniques are similar to DeepCS [18] with only a difference in choosing the embedding architecture. For example, to capture the semantics of other aspects of the code snippet, MMAN [62] integrates multiple embedding networks (i.e., LSTM, Tree-LSTM and GGNN) to capture semantics of multiple aspects, such as Token, AST, and CFG. CodeBERT [13], CoaCor [66], and baselines in CodeSearchNet Challenge [25] only treat the code snippet as plain text (token sequence), which miss richer information such as APIs, AST, and CFG, etc. TBCNN [46] is a tree-based convolutional neural network that encodes the AST of the code snippet. Our baseline MMAN has encoded AST using tree-based neural networks and is inferior to our TranCS. All these works have a similar idea that first transforms both code snippets and queries into embedding representations into a unified embedding space with two encoders, and then measures the cosine similarity of these embedding representations. However, TranCS differs from previous work in two major dimensions: 1) TranCS first translates the code snippet into semantic-preserving natural language descriptions. In this case, the generated translations and comments are homogeneous. 2) Based on code translation, TranCS naturally uses a shared word mapping mechanism, which can produce consistent embeddings for the same words, thereby better capturing the shared semantic information of translations and comments.

9 CONCLUSION

In this paper, we propose a context-aware code translation technique, which can translate code snippets into natural language descriptions with preserved semantics. In addition, we propose a shared word mapping mechanism to produce consistent embeddings for the same words/tokens in comments and code snippets, so as to capture the shared semantic information. On the basis of context-aware code translation and shared word mapping, we implement a novel code search technique TranCS. We conduct comprehensive experiments to evaluate the effectiveness of TranCS, and experimental results show that TranCS is an effective CS technique and substantially outperforms the state-of-the-art techniques.

In future work, we will further explore the following two dimensions: (1) as shown in Figure10, statistical results on large-scale data sets show that most code snippets have no more than 20 lines. Within this range, TranCS is robust and stable. Constructing representations of long code snippets is still an open problem, and we leave it to future work. (2) LSTM encoder is just a component of TranCS, which can be easily replaced with more advanced (including pre-trained) models in [6, 39]. We will explore more advanced models in future work.

ACKNOWLEDGEMENT

The authors would like to thank the anonymous reviewers for insightful comments. This work is supported partially by National Natural Science Foundation of China(61690201, 62141215).

REFERENCES

[1] Dzmitry Bahdanau, Kyunghyun Cho, and Yoshua Bengio. 2015. Neural Machine Translation by Jointly Learning to Align and Translate. In *Proceedings of the 3th International Conference on Learning Representations*. San Diego, CA, USA, 1–15.

[2] Yoshua Bengio, Patrice Y. Simard, and Paolo Frasconi. 1994. Learning long-term dependencies with gradient descent is difficult. *IEEE Transactions on Neural Networks* 5, 2 (1994), 157–166.

[3] Daniel Bilar. 2007. Opcodes as predictor for malware. *International Journal of Electronic Security and Digital Forensics* 1, 2 (2007), 156–168.

[4] Joel Brandt, Mira Dontcheva, Marcos Weskamp, and Scott R. Klemmer. 2010. Example-centric programming: integrating web search into the development environment. In *Proceedings of the 28th International Conference on Human Factors in Computing Systems*. ACM, Atlanta, Georgia, USA, 513–522.

[5] Joel Brandt, Philip J. Guo, Joel Lewenstein, Mira Dontcheva, and Scott R. Klemmer. 2009. Two studies of opportunistic programming: interleaving web foraging, learning, and writing code. In *Proceedings of the 27th International Conference on Human Factors in Computing Systems*. ACM, Boston, MA, USA, 1589–1598.

[6] Nghi D. Q. Bui, Yijun Yu, and Lingxiao Jiang. 2021. Self-Supervised Contrastive Learning for Code Retrieval and Summarization via Semantic-Preserving Transformations. In *Proceedings of the 44th International ACM SIGIR Conference on Research and Development in Information Retrieval*. ACM, Virtual Event, Canada, 511–521.

[7] José Cambronero, Hongyu Li, Seohyun Kim, Koushik Sen, and Satish Chandra. 2019. When deep learning met code search. In *Proceedings of the 13 Joint Meeting on European Software Engineering Conference and Symposium on the Foundations of Software Engineering*. ACM, Tallinn, Estonia, 964–974.

[8] Claudio Carpineto and Giovanni Romano. 2012. A Survey of Automatic Query Expansion in Information Retrieval. *Comput. Surveys* 44, 1 (2012), 1–50.

[9] CodeSearchNet. 2019. CodeSearchNet Data. site: https://github.com/github/CodeSearchNet#data. Accessed: 2021.

[10] Ronan Collobert, Jason Weston, Léon Bottou, Michael Karlen, Koray Kavukcuoglu, and Pavel P. Kuksa. 2011. Natural Language Processing (Almost) from Scratch. *Journal of Machine Learning Research* 12, ARTICLE (2011), 2493–2537.

[11] Yuxin Ding, Wei Dai, Shengli Yan, and Yumei Zhang. 2014. Control flow-based opcode behavior analysis for Malware detection. *Computers & Security* 44 (2014), 65–74.

[12] Chunrong Fang, Zixi Liu, Yangyang Shi, Jeff Huang, and Qingkai Shi. 2020. Functional code clone detection with syntax and semantics fusion learning. In *Proceedings of the 29th International Symposium on Software Testing and Analysis*. ACM, Virtual Event, USA, 516–527.

[13] Zhangyin Feng, Daya Guo, Duyu Tang, Nan Duan, Xiaocheng Feng, Ming Gong, Linjun Shou, Bing Qin, Ting Liu, Daxin Jiang, and Ming Zhou. 2020. CodeBERT: A Pre-Trained Model for Programming and Natural Languages. In *Proceedings of the 25th Conference on Empirical Methods in Natural Language Processing: Findings*. Association for Computational Linguistics, Online Event, 1536–1547.

[14] Andrea Frome, Gregory S. Corrado, Jonathon Shlens, Samy Bengio, Jeffrey Dean, Marc'Aurelio Ranzato, and Tomás Mikolov. 2013. DeViSE: A Deep Visual-Semantic Embedding Model. In *proceedings of the 27th Annual Conference Neural Information Processing Systems*. Curran Associates Inc., Lake Tahoe, Nevada, United States, 2121–2129.

[15] Ali Ghanbari, Samuel Benton, and Lingming Zhang. 2019. Practical program repair via bytecode mutation. In *Proceedings of the 28th International Symposium on Software Testing and Analysis*. ACM, Beijing, China, 19–30.

[16] Mohammad Gharehyazie, Baishakhi Ray, and Vladimir Filkov. 2017. Some from here, some from there: cross-project code reuse in GitHub. In *Proceedings of the 14th International Conference on Mining Software Repositories*. IEEE Computer Society, Buenos Aires, Argentina, 291–301.

[17] Inc. GitHub. 2008. GitHub. site: https://github.com. Accessed: 2021.

[18] Xiaodong Gu, Hongyu Zhang, and Sunghun Kim. 2018. Deep code search. In *Proceedings of the 40th International Conference on Software Engineering*. ACM, Gothenburg, Sweden, 933–944.

[19] Xiaodong Gu, Hongyu Zhang, Dongmei Zhang, and Sunghun Kim. 2016. Deep API learning. In *Proceedings of the 24th International Symposium on Foundations of Software Engineering*. ACM, Seattle, WA, USA, 631–642.

[20] Piyush Gupta, Nikita Mehrotra, and Rahul Purandare. 2020. JCoffee: Using Compiler Feedback to Make Partial Code Snippets Compilable. In *Proceedings of the 36th International Conference on Software Maintenance and Evolution*. IEEE, Adelaide, Australia, 810–813.

[21] Rajarshi Haldar, Lingfei Wu, Jinjun Xiong, and Julia Hockenmaier. 2020. A Multi-Perspective Architecture for Semantic Code Search. In *Proceedings of the 58th Annual Meeting of the Association for Computational Linguistics*. Association for Computational Linguistics, Online, 8563–8568.

[22] Emily Hill, Lori L. Pollock, and K. Vijay-Shanker. 2009. Automatically capturing source code context of NL-queries for software maintenance and reuse. In *Proceedings of the 31st International Conference on Software Engineering*. IEEE, Vancouver, Canada, 232–242.

[23] Sepp Hochreiter and Jürgen Schmidhuber. 1997. Long Short-Term Memory. *Neural Computation* 9, 8 (1997), 1735–1780.

[24] Xing Hu, Ge Li, Xin Xia, David Lo, and Zhi Jin. 2018. Deep code comment generation. In *Proceedings of the 26th International Conference on Program Comprehension*. ACM, Gothenburg, Sweden, 200–210.

[25] Hamel Husain, Ho-Hsiang Wu, Tiferet Gazit, Miltiadis Allamanis, and Marc Brockschmidt. 2019. CodeSearchNet Challenge: Evaluating the State of Semantic Code Search. *CoRR* abs/1909.09436 (2019), 1–6.

[26] Stack Exchange Inc;. 2008. Stack Overflow. site: https://stackoverflow.com/. Accessed: 2021.

[27] Iman Keivanloo, Juergen Rilling, and Ying Zou. 2014. Spotting working code examples. In *Proceedings of the 36th International Conference on Software Engineering*. ACM, Hyderabad, India, 664–675.

[28] Mert Kilickaya, Aykut Erdem, Nazli Ikizler-Cinbis, and Erkut Erdem. 2017. Re-evaluating Automatic Metrics for Image Captioning. In *Proceedings of the 15th Conference of the European Chapter of the Association for Computational Linguistics*. Association for Computational Linguistics, Valencia, Spain, 199–209.

[29] Diederik P. Kingma and Jimmy Ba. 2015. Adam: A Method for Stochastic Optimization. In *Proceedings of the 3th International Conference on Learning Representations – Poster*. OpenReview.net, San Diego, CA, USA, 1–15.

[30] Quoc V. Le and Tomás Mikolov. 2014. Distributed Representations of Sentences and Documents. In *Proceedings of the 31th International Conference on Machine Learning*. JMLR.org, Beijing, China, 1188–1196.

[31] Otávio Augusto Lazzarini Lemos, Adriano Carvalho de Paula, Felipe Capodifoglio Zanichelli, and Cristina Videira Lopes. 2014. Thesaurus-based automatic query expansion for interface-driven code search. In *Proceedings of the 11th Working Conference on Mining Software Repositories*. ACM, Hyderabad, India, 212–221.

[32] Yujia Li, Daniel Tarlow, Marc Brockschmidt, and Richard S. Zemel. 2016. Gated Graph Sequence Neural Networks. In *Proceedings of the 4th International Conference on Learning Representations*. OpenReview.net, San Juan, Puerto Rico, 1–20.

[33] Tim Lindholm, Frank Yellin, Gilad Bracha, Alex Buckley, and Daniel Smith. 2021. The Java Virtual Machine Specification. site: https://docs.oracle.com/javase/specs/jvms/se8/html/index.html. Accessed: 2021.

[34] Tim Lindholm, Frank Yellin, Gilad Bracha, Alex Buckley, and Daniel Smith. 2021. The Java Virtual Machine Specification-Local Variables. site: https://docs.oracle.com/javase/specs/jvms/se8/html/jvms-2.html#jvms-2.6. Accessed: 2021.

[35] Chao Liu, Xin Xia, David Lo, Cuiyun Gao, Xiaohu Yang, and John Grundy. 2020. Opportunities and Challenges in Code Search Tools. *CoRR* abs/2011.02297 (2020), 1–35.

[36] Meili Lu, Xiaobing Sun, Shaowei Wang, David Lo, and Yucong Duan. 2015. Query expansion via WordNet for effective code search. In *Proceedings of the 22nd International Conference on Software Analysis, Evolution, and Reengineering*. IEEE Computer Society, Montreal, QC, Canada, 545–549.

[37] Fei Lv, Hongyu Zhang, Jian-Guang Lou, Shaowei Wang, Dongmei Zhang, and Jianjun Zhao. 2015. CodeHow: Effective Code Search Based on API Understanding and Extended Boolean Model (E). In *Proceedings of the 30th International Conference on Automated Software Engineering*. IEEE Computer Society, Lincoln, NE, USA, 260–270.

[38] Robert C Martin. 2009. *Clean code: a handbook of agile software craftsmanship*. Pearson Education.

[39] Antonio Mastropaolo, Simone Scalabrino, Nathan Cooper, David Nader-Palacio, Denys Poshyvanyk, Rocco Oliveto, and Gabriele Bavota. 2021. Studying the Usage of Text-To-Text Transfer Transformer to Support Code-Related Tasks. In *Proceedings of the 43rd International Conference on Software Engineering*. IEEE, Madrid, Spain, 336–347.

[40] Collin McMillan, Mark Grechanik, Denys Poshyvanyk, Qing Xie, and Chen Fu. 2011. Portfolio: finding relevant functions and their usage. In *Proceedings of the 33rd International Conference on Software Engineering*. ACM, Waikiki, Honolulu , HI, USA, 111–120.

[41] Collin McMillan, Negar Hariri, Denys Poshyvanyk, Jane Cleland-Huang, and Bamshad Mobasher. 2012. Recommending source code for use in rapid software prototypes. In *Proceedings of the 34th International Conference on Software Engineering*. IEEE Computer Society, Zurich, Switzerland, 848–858.

[42] Tomás Mikolov, Kai Chen, Greg Corrado, and Jeffrey Dean. 2013. Efficient Estimation of Word Representations in Vector Space. In *Proceedings of the 1st International Conference on Learning Representations, Workshop Track*. OpenReview.net, Scottsdale, Arizona, USA, 1–12.

[43] Tomas Mikolov, Ilya Sutskever, Kai Chen, Gregory S. Corrado, and Jeffrey Dean. 2013. Distributed Representations of Words and Phrases and their Compositionality. In *Proceedings of the 27th Annual Conference on Neural Information Processing Systems*. Curran Associates Inc., Lake Tahoe, Nevada, United States, 3111–3119.

[44] George A. Miller. 1995. WordNet: A Lexical Database for English. *Commun. ACM* 38, 11 (1995), 39–41.

[45] Robert Moskovitch, Clint Feher, Nir Tzachar, Eugene Berger, Marina Gitelman, Shlomi Dolev, and Yuval Elovici. 2008. Unknown Malcode Detection Using OPCODE Representation. In *Proceedings of the First European Conference on Intelligence and Security Informatics*. Springer, Esbjerg, Denmark, 204–215.

[46] Lili Mou, Ge Li, Lu Zhang, Tao Wang, and Zhi Jin. 2016. Convolutional Neural Networks over Tree Structures for Programming Language Processing. In *Proceedings of the 30th Conference on Artificial Intelligence*. AAAI Press, Phoenix, Arizona, USA, 1287–1293.

[47] Tam The Nguyen, Hung Viet Pham, Phong Minh Vu, and Tung Thanh Nguyen. 2016. Learning API usages from bytecode: a statistical approach. In *Proceedings of the 38th International Conference on Software Engineering*. ACM, Austin, TX, USA, 416–427.

[48] Liming Nie, He Jiang, Zhilei Ren, Zeyi Sun, and Xiaochen Li. 2016. Query Expansion Based on Crowd Knowledge for Code Search. *IEEE Transactions on Services Computing* 9, 5 (2016), 771–783.

[49] An Examination of Software Engineering Work Practices. 1997. Janice Singer and Timothy C. Lethbridge and Norman G. Vinson and Nicolas Anquetil. In *Proceedings of the 7th conference of the Centre for Advanced Studies on Collaborative Research*. IBM, Toronto, Ontario, Canada, 174–188.

[50] Hamid Palangi, Li Deng, Yelong Shen, Jianfeng Gao, Xiaodong He, Jianshu Chen, Xinying Song, and Rabab K. Ward. 2015. Deep Sentence Embedding Using the Long Short Term Memory Network: Analysis and Application to Information Retrieval. *CoRR* abs/1502.06922 (2015), 1–15.

[51] Denys Poshyvanyk, Maksym Petrenko, Andrian Marcus, Xinrong Xie, and Dapeng Liu. 2006. Source Code Exploration with Google. In *Proceedings of the 22nd International Conference on Software Maintenance*. IEEE Computer Society, Philadelphia, Pennsylvania, USA, 334–338.

[52] Mohammad Masudur Rahman and Chanchal K. Roy. 2018. Effective Reformulation of Query for Code Search Using Crowdsourced Knowledge and Extra-Large Data Analytics. In *Proceedings of the 34th International Conference on Software Maintenance and Evolution*. IEEE Computer Society, Madrid, Spain, 473–484.

[53] Saksham Sachdev, Hongyu Li, Sifei Luan, Seohyun Kim, Koushik Sen, and Satish Chandra. 2018. Retrieval on source code: a neural code search. In *Proceedings of the 2nd International Workshop on Machine Learning and Programming Languages*. ACM, Philadelphia, PA, USA, 31–41.

[54] Caitlin Sadowski, Kathryn T. Stolee, and Sebastian G. Elbaum. 2015. How developers search for code: a case study. In *Proceedings of the 10th Joint Meeting on European Software Engineering Conference and Symposium on the Foundations of Software Engineering*. ACM, Bergamo, Italy, 191–201.

[55] Hinrich Schütze, Christopher D Manning, and Prabhakar Raghavan. 2008. *Introduction to information retrieval*. Vol. 39. Cambridge University Press Cambridge.

[56] Jianhang Shuai, Ling Xu, Chao Liu, Meng Yan, Xin Xia, and Yan Lei. 2020. Improving Code Search with Co-Attentive Representation Learning. In *Proceedings of the 28th International Conference on Program Comprehension*. ACM, Seoul, Republic of Korea, 196–207.

[57] Fang-Hsiang Su, Jonathan Bell, Kenneth Harvey, Simha Sethumadhavan, Gail E. Kaiser, and Tony Jebara. 2016. Code relatives: detecting similarly behaving software. In *Proceedings of the 24th International Symposium on Foundations of Software Engineering*. ACM, Seattle, WA, USA, 702–714.

[58] Weisong Sun and Yuchen Chen. 2022. Source Code and Dataset of TranCS. site: https://github.com/wssun/TranCS. Accessed: 2022.

[59] Kai Sheng Tai, Richard Socher, and Christopher D. Manning. 2015. Improved Semantic Representations From Tree-Structured Long Short-Term Memory Networks. In *Proceedings of the 53rd Annual Meeting of the Association for Computational Linguistics*. The Association for Computer Linguistics, Beijing, China, 1556–1566.

[60] Michele Tufano, Cody Watson, Gabriele Bavota, Massimiliano Di Penta, Martin White, and Denys Poshyvanyk. 2018. Deep learning similarities from different representations of source code. In *Proceedings of the 15th International Conference on Mining Software Repositories*. ACM, Gothenburg, Sweden, 542–553.

[61] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, undefinedukasz Kaiser, and Illia Polosukhin. 2017. Attention is All You Need. In *Proceedings of the 31st Annual Conference on Neural Information Processing Systems*. Curran Associates Inc., Long Beach, CA, USA, 5998–6008.

[62] Yao Wan, Jingdong Shu, Yulei Sui, Guandong Xu, Zhou Zhao, Jian Wu, and Philip S. Yu. 2019. Multi-modal Attention Network Learning for Semantic Source Code Retrieval. In *Proceedings of the 34th International Conference on Automated Software Engineering*. IEEE, San Diego, CA, USA, 13–25.

[63] Xin Xia, Lingfeng Bao, David Lo, Pavneet Singh Kochhar, Ahmed E. Hassan, and Zhenchang Xing. 2017. What do developers search for on the web? *Empirical Software Engineering* 22, 6 (2017), 3149–3185.

[64] Ling Xu, Huanhuan Yang, Chao Liu, Jianhang Shuai, Meng Yan, Yan Lei, and Zhou Xu. 2021. Two-Stage Attention-Based Model for Code Search with Textual and Structural Features. In *Proceedings of the 28th International Conference on Software Analysis, Evolution and Reengineering*. IEEE, Honolulu, HI, USA, 342–353.

[65] Yinxing Xue, Zhengzi Xu, Mahinthan Chandramohan, and Yang Liu. 2019. Accurate and Scalable Cross-Architecture Cross-OS Binary Code Search with Emulation. *IEEE Transactions on Software Engineering* 45, 11 (2019), 1125–1149.

[66] Ziyu Yao, Jayavardhan Reddy Peddamail, and Huan Sun. 2019. CoaCor: Code Annotation for Code Retrieval with Reinforcement Learning. In *Proceedings of the 28th The World Wide Web Conference*. ACM, San Francisco, CA, USA, 2203–2214.

[67] Chen Zeng, Yue Yu, Shanshan Li, Xin Xia, Zhiming Wang, Mingyang Geng, Bailin Xiao, Wei Dong, and Xiangke Liao. 2021. deGraphCS: Embedding Variable-based Flow Graph for Neural Code Search. *CoRR* abs/2103.13020 (2021), 1–21.

[68] Qihao Zhu, Zeyu Sun, Xiran Liang, Yingfei Xiong, and Lu Zhang. 2020. OCoR: An Overlapping-Aware Code Retriever. In *Proceedings of the 35th International Conference on Automated Software Engineering*. IEEE, Melbourne, Australia, 883–894.